

DD2356-Project-Electromagnetic

This application simulates electromagnetic wave propagation using the Finite-Difference Time-Domain (FDTD) method.

Contribution

- **Serban-Mihai Ionescu:** Baseline C/C++ Implementation, OpenMP Parallelization, Hybrid Optimizations
- **Hieu Vu:** MPI Parallelization
- **Theodor Dan Popescu:** Hybrid OpenMP + MPI

1. Electromagnetics Test Case for Correctness

Test Case Description

- **Initial Condition:** The electric field is initialized with a Gaussian pulse centered at $x = N_X \times DX/2$, while the magnetic field starts at zero.
- **Expected Behavior:** With simple absorbing boundaries, the Gaussian pulse splits into two pulses propagating outward at numerical speed $c \approx DX/DT$. After N_{STEPS} , the peak is expected to shift roughly $N_{STEPS}/2$ grid points in each direction.

Verification Steps

1. **Visual Inspection:** We modified the serial code to save the electric field in a matrix at various instances. The data is written to files and plotted using a Python script, confirming the symmetric split.
2. **Peak Position Check:** We input a field much larger than the distance the wave is meant to travel to prevent "reflection" artifacts. We compare the index of the maximum electric field at the final time step with the mathematical expectation.
3. **Quantitative Comparison:** The simulated peak position correctly matches the analytical prediction. In the case of deviation, the relative error is computed as $\frac{abs(expected - actual)}{expected}$ and must remain within acceptable bounds (e.g., $<5\%$).

2. Baseline C/C++ Implementation

Performance Profiling & Metrics

We profiled the baseline implementation across three systems (Personal Computer, Dardel Supercomputer, and the School Cluster) using the `perf` tool and `time` utility.

Small Problem Size ($N_X=400$, $N_{STEPS}=1000$):

- **PC:** Executes in $\sim 0.005s$. With $\sim 6KB$ of data, everything fits inside the L1 cache, yielding very few L1 load misses. The 4.35 instructions per cycle (insn/cycle) means the code is highly pipelined.
- **Big Boy Dardel:** Executes in $\sim 0.008s$. Shows lower instruction throughput (1.71 insn/cycle), indicating the supercomputer does not automatically apply the same default pipelining optimizations without flags.
- **School Cluster:** Executes in $\sim 0.003s$ with excellent pipelining (3.30 insn/cycle) and a massive memory bandwidth throughput of $\sim 5.8GB/s$.

Large Problem Size ($N_X=80000$, $N_{STEPS}=20000$):

- **PC:** Executes in $\sim 5.7s$. The L1 data cache misses are high (~ 400 million), which is mathematically expected: arrays exceed cache line limits, resulting in $\sim 20,000$ misses per step. Memory accessed hits $\sim 14GB/s$.
- **Big Boy Dardel:** Executes in $\sim 11.2s$. We observed an amazing amount of page misses, pointing to the OS architecture (potentially utilizing very large memory pages).
- **School Cluster:** Executes in $\sim 4.9s$. *Holly pipelining!* The cluster maximizes its 6-step pipeline process, hitting 5.58 insn/cycle and processing memory at an astonishing 28GB/s. Pre-fetching by the OS keeps TLB load misses near 0.

Across all systems, the `perf` hotspots clearly show that `update_E` and `update_H` account for nearly 100% of the active compute time (once file I/O overhead is removed).

System	Problem Size	Time (s)	Cycles	Instructions	Insn/Cycle	L1 Cache Loads	Cache Misses	dTLB Loads	dTLB Misses
Personal Computer	Small (400x1K)	0.005	7,832,944	34,099,260	4.35	7,212,485	119,053	2,648	789
Dardel Supercomputer	Small (400x1K)	0.008	13,040,683	22,248,016	1.71	14,799,274	26,581	13,768	2,019
School Cluster	Small (400x1K)	0.003	8,511,330	28,079,302	3.30	11,655,339	14,030	11,658,172	1,076
Personal Computer	Large (80Kx20K)	5.767	23,856,689,210	105,676,455,719	4.43	28,995,750,517	21,641,634	12,837,658	36,878
Dardel Supercomputer	Large (80Kx20K)	11.264	35,905,756,922	70,412,692,558	1.96	51,222,765,885	13,175,407	12,569,313	4,801

System	Problem Size	Time (s)	Cycles	Instructions	Insn/Cycle	L1 Cache Loads	Cache Misses	dTLB Loads	dTLB Misses
School Cluster	Large (80Kx20K)	4.997	18,942,826,723	105,621,143,093	5.58	44,805,895,857	144,280	44,806,070,655	4,621

Upper Bound of Parallel Speedup

Using Amdahl's Law, we approximated the maximum theoretical speedup ($1/f$, where f is the serial fraction of the code):

- **Small Grid (\$NX=400\$):** Serial fraction $f \approx 0.073$. Upper bound speedup: **13.69x**.
- **Large Grid (\$NX=80000\$):** Serial fraction $f \approx 0.000122$. Upper bound speedup: **8196.72x**. (We can expect an excellent speedup here).

Computing System	Threads	Execution Time (s)	Speedup
Local Machine	1 (Serial)	5.318	1.00x
	8	0.847	6.27x
	16	0.855	6.22x
Dardel Supercomputer	1 (Serial)	10.662	1.00x
	4	3.010	3.56x
	8	2.500	4.26x
School Cluster	1 (Serial)	5.209	1.00x
	8	0.879	5.93x
	16	0.550	9.47x

3. OpenMP Parallelization

Compute-Intensive Parts & Implementation

Statically and dynamically, `update_E` and `update_H` are the compute bottlenecks. Initial profiling showed `fprintf` taking 54% of the time, but after precompilation techniques hid the file I/O, the updates dominated 99% of the execution.

Because `update_E` dynamically depends on `update_H`, we cannot parallelize the outer time-step loop. We instead parallelized the inner spatial loop using `#pragma omp parallel for schedule(static)` since operations take a uniform amount of time per index. Verification showed identical relative error ($\$0.012\%$) between serial and OpenMP versions.

Communication Overhead & Scalability

In shared memory, $T_{\text{parallel}} = T_{\text{compute}} / P + T_{\text{overhead}}$. The overhead stems from thread synchronization (the implicit barriers at the end of every `pragma omp for`) and the cost of thread creation/destruction.

- Because each thread writes strictly to its designated indices, we effectively avoid cache coherence penalties (false sharing).
- However, with increasing steps, the constant barrier overhead per step heavily impacts scalability. Testing an alternative structure—spawning threads only once outside the time loop—actually increased execution time by 0.1s.

Parallel Speedup Evaluation

Comparing the large grid (\$NX=80000, NSTEPS=20000) against the serial version:

- **PC (8 threads):** 0.847s (Speedup: **6.27x**). *Great success!*
- **Dardel (8 threads):** 2.50s (Speedup: **4.26x**).
- **School Cluster (16 threads):** 0.55s (Speedup: **9.47x**). Past 32 threads, performance degraded.

When comparing metrics, the OpenMP version shows 3x higher overall cache misses than serial. Distributing the work across more CPUs pushes data down to lower, shared cache levels (L2/L3), creating a higher aggregate miss ratio despite the compute speedup.

System	Version	Problem Size	Time (s)	Cycles	Instructions	Insn/Cycle	L1 Cache Loads	Cache Misses	dTLB Loads	dTLB Misses
Personal Computer	OpenMP (8 Threads)	Large (80Kx20K)	0.892	26,106,256,491	106,051,143,632	4.06	28,985,879,781	32,821,478	1,257,951	18,613

4. MPI Parallelization

Compute-Intensive Parts & Implementation

We implemented pure message passing (MPI) to scale across multiple nodes. The global grid (\$NX\$) is divided into local subgrids across \$P\$ processes.

- **Grid Partitioning & Ghost Cells:** Each local array has two extra indices (halos) for boundary data from neighboring processes.
- **Data Swapping:** Using `MPI_Sendrecv`, ranks exchange their edge values (e.g., `E[1]` sent left, `H[N_{local}]` sent right). This bidirectional shift prevents deadlocks and correctly handles cross-boundary dependencies.

Verification

We validated the domain decomposition by running the simulation up to 64 processes. The local peak detection results were gathered at Rank 0 using `MPI_Gather`. The exact matching of the peak index at `49994.00` across all parallel sweeps verifies that the message passing implementation introduces zero physical distortion.

Communication Overhead & Scalability

Unlike OpenMP where threads share memory, MPI processes have separate address spaces.

- **Communication Cost:** Small payloads (2 doubles per direction per step) keep direct communication overhead very low.
- **Synchronization Cost:** `MPI_Sendrecv` acts as an implicit barrier. As `SP` increases and local computation per step decreases, communication latency starts to dominate, especially when crossing NUMA socket boundaries.

Parallel Speedup Evaluation

We executed the benchmark locally, on the Dardel supercomputer, and on the school cluster for a large grid (`SNX = 80000` and `NSTEPS = 20000`) using `SP` processes.

Local Computer (PC)

Processes (<code>Sp</code>)	Execution Time (<code>ST_p</code>)	Speedup (<code>SS</code>)	Efficiency (<code>SE</code>)
1	5.2400 s	1.00x	100.0%
2	2.6805 s	1.95x	97.8%
4	1.9095 s	2.74x	68.6%
8	1.8141 s	2.89x	36.1%
16	1.3826 s	3.79x	23.7%

Dardel Supercomputer

Processes (<code>Sp</code>)	Execution Time (<code>ST_p</code>)	Speedup (<code>SS</code>)	Efficiency (<code>SE</code>)
1	9.2084 s	1.00x	100.0%
2	4.8286 s	1.91x	95.5%
4	2.4500 s	3.76x	94.0%
8	1.1829 s	7.78x	97.3%
16	0.6232 s	14.78x	92.4%
32	0.5973 s	15.42x	48.2%
64	0.2284 s	40.33x	63.0%

School Cluster

Processes (<code>Sp</code>)	Execution Time (<code>ST_p</code>)	Speedup (<code>SS</code>)	Efficiency (<code>SE</code>)
1	6.5394 s	1.00x	100.0%
2	3.2343 s	2.02x	101.1%
4	1.7871 s	3.66x	91.5%
8	1.0835 s	6.04x	75.5%
16	1.2243 s	5.34x	33.4%

Comparison & Analysis

- **Local Machine:** The scaling degrades quickly past 4 processes due to the physical core limit. Oversubscription introduces significant OS-level scheduling overhead, stalling all processes during `MPI_Sendrecv`.
- **Dardel Supercomputer:** Efficiency stays above 92% up to 16 cores. At 32 cores, efficiency falls to 48% due to NUMA architecture bottlenecks (crossing Core Complex boundaries increases latency). However, at 64 cores, the massive compute capability recovers the scaling to a **40.33x** speedup.
- **School Cluster:** Reaches a peak **6.04x** speedup on 8 processes. At 16 processes, execution time actually increases, suggesting a hard limit of 8 physical cores on the node.

- **Comparison to OpenMP:** MPI scaled better than OpenMP (e.g., 7.78x vs 4.26x on Dardel for 8 cores) because separate address spaces avoid cache coherence penalties and false sharing inherent to shared memory.

5. Hybrid OpenMP + MPI

Design and Implementation

The design for the hybrid OpenMP + MPI version was taken primarily from the design of the individual versions, with an added optimization regarding halo exchange. With now (N,P) threads and processes running the code in parallel, we increase the domain partitioning even further. Now, we have P processes take ownership for a portion of the large array, while the N threads running on each processes will further partition each process' array region. Individual threads for a process do not need to share their chunk boundary values as the overarching process array chunk is shared between them.

```
void update_H_interior(double *E, double *H, int local_NX) {
    #pragma omp for simd schedule(static)
    for (int i = 2; i < local_NX - 2; i++) {
        H[i] = H[i] + (DT / DX) * (E[i + 1] - E[i]);
    }
}
```

However, similar to the MPI version, the different processes themselves do need to communicate their updated boundary values between each other, as they do not have access to the updates done to boundary values on other chunks. A halo exchange is implemented to have the processes share these values with each other, with the left and right most values of the allocated array for each process being allocated to store those halo values.

The halo exchanged is altered from the pure MPI approach, as the halo exchange communication between processes now overlaps with the interior array processing for each process. This is possible due to the non-boundary values of the array not relying on the halo values, and so their processing can proceed without impact, while waiting for the receipt of the neighboring processes halo values. This added change might allow for faster speedups when the computation per thread is still quite high, and the communication overhead for halo values can be fully overlapped by the computation overhead.

```
#pragma omp parallel
{
    for (int t = 0; t < NSTEPS; t++) {
        #pragma omp single
        {
            nreqs = 0;
            if (rank > 0) MPI_Irecv(&E[0], 1, MPI_DOUBLE, rank-1, 1, MPI_COMM_WORLD, &nreqs);
            if (rank < size-1) MPI_Irecv(&H[local_NX-1], 1, MPI_DOUBLE, rank+1, 0, MPI_COMM_WORLD, &nreqs);
            if (rank < size-1) MPI_Isend(&H[local_NX-2], 1, MPI_DOUBLE, rank+1, 1, MPI_COMM_WORLD, &nreqs);
            if (rank > 0) MPI_Isend(&E[1], 1, MPI_DOUBLE, rank-1, 0, MPI_COMM_WORLD, &nreqs);

            update_H_interior(E, H, local_NX);

            #pragma omp single
            {
                MPI_Waitall(nreqs, reqs, MPI_STATUSES_IGNORE);
                update_H_boundary(E, H, local_NX, rank, size);
            }
        }
    }
}
```

Verification

Verification was done similarly to the OpenMP and MPI versions. The peak detection was MPI_Gathered at rank 0 to find the global maximum, and when testing on the same NX and step counts, the peak index matched the sequential version on all (N,P) thread/process counts:

We also verified by running the hybrid code with plotting enabled, and when visualizing the plotting with our visualize.py script, we can see the correct splitting of the E field identically to the sequential version (plots are stored in plots/field_plot_hybrid and plots/field_plot_serial).

Communication Modeling

The main communication overhead in our system comes from the halo exchange done by neighboring MPI processes, and from thread synchronization. The OpenMP communication overhead in the hybrid version is identical to the base OpenMP version, as we have no inter-thread communication cost. We do have thread synchronization costs due to the pragma single blocks for the halo exchange and boundary computation, and for the two parallel for loops for the updating H and E field interior values. Each single block and for loop has an implicit barrier at exit where all threads must wait, and so for each time step, we incur two thread barriers for the two field interior value computations, and 4 thread barriers for the 4 pragma single blocks (a halo exchange and boundary computation for both H and E fields). OMP thread barrier overhead scales logarithmically with thread count, and so we have $6\log(N)$ as our estimate for thread synchronization costs.

The main communication overhead we have though is from the MPI halo exchange. For each neighboring processes, for both the H and E fields, we need to send and receive 2 halo values (only one for the process with rank 0 and rank # processes - 1), at each step. Therefore, at each step, for P processes, we have $4*(P-1)$ MPI_Irecv and $4*(P-1)$ MPI_Isend, for a total of $8*(P-1)$ MPI communications every time step.

In total for each time step, for (N,P) threads and processes, we have a model of the communication costs of $8*(P-1)$ MPI_Isend/Irecv + $6\log(N)$ thread barriers. This affects our scalability, because as we grow the process/thread count, the added domain segmentation benefit for computation performance gets smaller. Meanwhile, we have a linearly scaling process communication overhead and a logarithmically scaling thread synchronization overhead, which past a certain (N,P) count, will start to dominate the added parallelization benefits and will degrade performance. This limits the upper level of our scalability that will still show performance improvement.

Results

To see how the hybrid OpenMP + MPI code performs, we fixed our number of cores ($N \times P$ for N threads and P processes) to values of 2,4,8, and 16. On each total $N \times P$, we tested a number of different $\langle N, P \rangle$ configurations to see which was optimal, and to observe any trends as we increased our $N \times P$ core total:

Total ($N \times P$)	Configs (N, P) → Strong Scaling Speedup				
2	(2,1) → 1.02	(1,2) → 1.96			
4	(4,1) → 0.60	(1,4) → 3.25	(2,2) → 2.03		
8	(8,1) → 0.34	(1,8) → 4.33	(4,2) → 0.92	(2,4) → 3.58	
16	(16,1) → N/A	(1,16) → 2.60	(8,2) → 0.37	(2,8) → 1.96	(4,4) → 2.31

We can observe that for all of our fixed core totals - the optimal configuration was to maximize P and only have one thread for each process. Additionally, all the lowest performing configurations were those where only one process was active, with all of the parallelization coming from OpenMP threads. This points to the barriers present in the OpenMP sections being a dominant factor and outweighing the parallelization benefit. On the other hand, we see the large strong scaling benefits when increasing the number of processes, with a large speedup increase for all process counts when each only runs one thread. We still notice the slight impact of the MPI Halo exchange, as we see that increasing from (1,8) to (1,16), and from (2,4) to (2,8), while the performance is still better than a sequential execution, the speedup decreases following the process count increase. Overall, we notice that the OpenMP thread synchronization seems to outweigh the parallelism benefits, while the MPI domain segmentation is able to provide much large speedup benefits up to a certain point.

Hybrid Optimizations

All optimizations were targeted at a configuration of 4 nodes, 4 MPI processes, and 8 threads.

- **First optimization:** using `nowait` for the `for` parallelization.
 - This shaved off 0.04 seconds (0.83 to 0.79).
 - This works because we are doing synchronization fewer times.
- **Second optimization:** defining `#define DTDX (DT / DX)` from the start so that this operation is not done every time.
 - This resulted in higher time: 1.2 seconds.
 - This might just be an artefact, as it doesn't make any sense to have lower instructions but higher time.
- **Third optimization:** aligning the memory so that when it is fetched from RAM, we are not wasting space for random junk before the start of the array.
 - This didn't do much (1s version normal, 1.2 seconds version optimized).

GPU Offloading

Initially, GPU offloading failed compilation on the school cluster and Dardel partitions due to compiler issues and partition access.

However, we successfully resolved the compilation issues on the Ubuntu-based school cluster. The GNU compiler (GCC) was mistakenly passing host default security flags (`-fcf-protection=full` and `-fstack-protector-strong`) to the NVPTX GPU offload compiler during the Link-Time Optimization (LTO) phase, causing the assembler to crash.

By explicitly disabling these flags, we successfully compiled and ran the OpenMP target offload code:

```
cc -O3 -fopenmp -fcf-protection=none -fno-stack-protector -no-pie openMP_GPU.c -o gpu_fdt -lm
./gpu_fdt
```

```
jovyan@jupyter-hieuvtm:~/DD2356-Project-Electromagnetic$ cc -O2 -fopenmp -fcf-protection=none -fno-stack-protector openMP_GPU.c -o gpu_fdt -lm
/usr/bin/ld: /tmp/ccKfHrfQ.crtoffloadtable.o: warning: relocation against `__offload_vars_end' in read-only section `.rodata'
/usr/bin/ld: warning: creating DT_TEXTREL in a PIE
jovyan@jupyter-hieuvtm:~/DD2356-Project-Electromagnetic$ ./gpu_fdt
```

```
Expected position of the maximum electric field: 50000.000000
Actual position of the maximum electric field: 49994.000000
The maximum electric field is NOT at the expected position.
Relative error: 0.012000%
```

The execution yielded the exact expected numerical results (matching the CPU/serial run with a 0.012% relative error), proving that our GPU offloading implementation works correctly.